# MODELLING, ANALYSIS AND TRAJECTORY PLANNING OF A ROBOTIC MANIPULATOR

## INTRODUCTION

### OVERVIEW

Webots “is an open source and multi-platform desktop application used to simulate robots” (Cyberbotics, 2025) in a virtual environment . This document illustrates its usage in designing, implementing and testing a 3-degree-of-freedom RRR robotic manipulator. The project utilises a series of MATLAB .m files, a Webots World file, a series of PROTO models and a series of MATLAB Webots controllers.

The report is structured in order of the specified tasks, and a table of contents is included below:

## TABLE OF CONTENTS

# TASK 1 - DESIGN AND SIMULATION OF 3-DEGREE-OF-FREEDOM RRR ROBOT IN WEBOTS

The robotic manipulator in question is derived from three linked arms which act dependent on the previous link in the chain. The finalised model is shown below in Figure 1:

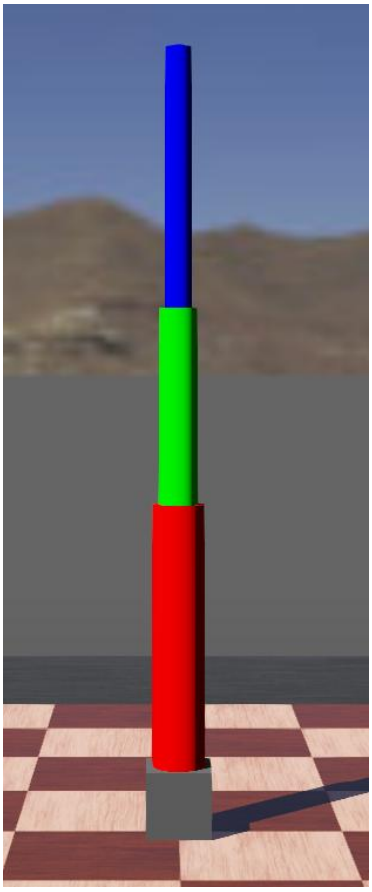

**Figure 1 - Designed Three Degree of Freedom, RRR Webots Robot**

The robot is situated in a 2x2 metre rectangular arena with a checkerboard pattern on the flat floor and short grey walls. In Webots, this allows for a suitable platform for which robot manipulation and testing can occur with minimal complications.

The robot stands atop a grey pedestal (designed as a 0.1x0.1x0.1 metre box) located at the bottom of the robot. This was included to prevent collision with the floor of the world in Webots. It is also useful for the robot in performing pick and place manoeuvres on objects in the world. This is explained in more detail in Task 5. The robot is constructed using three linked arms which are colour coded:

The first red link is designed as a cylinder with a radius of 4cm and height of 40cm. It rotates about the vertical Z axis in Webots, controlled by a Rotational Motor with 360 degrees of freedom situated at its base upon a Hinge Joint.

The second green link is similarly designed as a cylinder with a radius of 3cm and a height of 0.3. The radius is lowered to improve visual clarity and aesthetics, and its height is configured according to the project specifications. It rotates about the Y axis in Webots, controlled by a Rotational Motor with limited degrees of freedom (-60 degrees- 240 degrees) also situated at its base upon another Hinge Joint.

The third blue link is also similarly designed as a cylinder with a radius of 2cm and a height of 4m in accordance with the project specifications. It also rotates about the Y axis in Webots, controlled by a Rotational Motor with limited degrees of freedom (-60 degrees- 240 degrees) also situated at its base upon another Hinge Joint. As this is the end point of the robot (in this instance, without an object grabbing mechanism) a GPS node is connected to the end effector of the robot for later usage in identifying positional information of the robot.

# TASK 2 - FORWARD KINEMATICS IMPLEMENTATION USING DENAVIT-HARTENBERG (DH) CONVENTION

## THE DENAVIT-HARTENBERG METHOD

The Denavit-Hartenberg method is “one of the most important methods used in the process of designing robotic structures” (Prada, et al., 2020). In 1955, Jacques Denavit and Richard Hartenberg implemented and showcased this method to provide a way of describing the kinematic relationships between links connected by joints (Prada, et al., 2020).

The DH method is implemented in the project for the purpose of ascertaining the forward kinematic expression whereby we are interested in locating the position of the end effector when the robot is given three joint angles. The DH method for the robot can be derived using a set of DH parameters in the form of a matrix, which provides “a standard way to write the manipulator’s kinematic equation, especially for serial manipulators” (Prada, et al., 2020).

Initially we identify the joint angles. Subsequently, we assign coordinate frames where at each joint, the vertical offset, link length and twist angle are noted at each stage of the robot (link 1, link 2 and link 3 (end effector)). This is shown below in Table 1:

| *Link* | Joint Angles ($\theta_x$) | Vertical Offset ($d_x$) | Link Length($a_x$) | Twist Angle($\alpha_x$) |
|---|---|---|---|---|
| *1* | $\theta_1$ | 0.4 | 0 | 90 degrees |
| *2* | $\theta_2$ | 0 | 0.3 | 0 degrees |
| *3* | $\theta_3$ | 0 | 0.4 | 0 degrees |

**Table 1 - DH Table of Robot**

The DH parameters of the robot (given in Table 1) can then be inputted into the standard DH transformation matrix for each link to ascertain the position and orientation of one link with regards to another. The standard DH transformation matrix is given below in Figure 2:

$$Transform_i = \begin{bmatrix} \cos\theta_i & -\sin\theta_i\cos\alpha_i & \sin\theta_i\sin\alpha_i & a_i\cos\theta_i \\ \sin\theta_i & \cos\theta_i\cos\alpha_i & -\cos\theta_i\sin\alpha_i & a_i\sin\theta_i \\ 0 & \sin\alpha_i & \cos\alpha_i & d_i \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

**Figure 2 - Standard DH Transformation Matrix generated using LaTeX**

As such we implement the transformations for each link into MATLAB, the code for which is shown below in Figure 3:

## MATLAB IMPLEMENTATION

The transformation matrices are inputted according to the standard DH matrix shown above in Figure 2 and the parameters of the robot shown above in Table 1.

```
function End_Effector = forward_k(theta1, theta2, theta3)

%Correct theta 1 and 2
theta1 = theta1 + pi;
theta2 = theta2 + pi/2;

% Link Dimensions
d1 = 0.4;
a2 = 0.3;
a3 = 0.4;

% Pedestal Offset
ped_offset = [1 0 0 0;
          0 1 0 0;
          0 0 1 0.1;
          0 0 0 1];

% Transformation matrix: 0 - 1
Transform_01 = [cos(theta1), -sin(theta1)*cos(pi/2), sin(theta1)*sin(pi/2), 0*cos(theta1);
sin(theta1), cos(theta1)*cos(pi/2), -cos(theta1)*sin(pi/2), 0*sin(theta1);
0, sin(pi/2), cos(pi/2), d1;
0, 0, 0, 1];

% Transformation matrix: 1 - 2
Transform_12 = [cos(theta2), -sin(theta2)*cos(0), sin(theta2)*sin(0), a2*cos(theta2);
sin(theta2), cos(theta2)*cos(0), -cos(theta2)*sin(0), a2*sin(theta2);
0, sin(0), cos(0), 0;
0, 0, 0, 1];

% Transformation matrix: 2 - 3
Transform_23 = [cos(theta3), -sin(theta3)*cos(0), sin(theta3)*sin(0), a3*cos(theta3);
sin(theta3), cos(theta3)*cos(0), -cos(theta3)*sin(0), a3*sin(theta3);
0, sin(0), cos(0), 0;
0, 0, 0, 1];

% Final
End_Effector = ped_offset * Transform_01 * Transform_12 * Transform_23;
End_Effector = End_Effector(1:3, 4)
end
```

There are two notable additions to allow for calculations performed in MATLAB to match measured end effector positions showcased in Webots:

- The input theta values are altered to match the virtual environment of Webots by applying angle offsets to theta 1 and 2 specifically so that it matches that of the Webots implementation when the robot is at rest.
- A matrix which translates the end effector upwards in the z-axis by 0.1m is included to account for the pedestal with 0.1m height.

As a result, the end effector position is given in the form (x,y,z) and can be compared with the implementation in Webots.

**Figure 3 - Forward Kinematic Function in MATLAB using DH Transformation Matrices**

## WEBOTS IMPLEMENTATION

The Webots controller (“move_arm_by_degrees.m”) implementation of Task 1 simply ascertains the end effector position of the robot (using the previously mentioned GPS node) after the robot is given three radian angle positions for each rotational motor in each link. These angles are limited in accordance with the project parameters (360 degrees of freedom for theta 1, -60 - 240 degrees for thetas 2 and 3). Subsequently, the motor speed is set to 0.8 for visual clarity and an angle tolerance is set to when each motor reaches a position within 0.001 radians of the target. As such, small differences in the MATLAB and Webots implementations are expected. The motors move and have a time limit of 7.5 seconds to reach their

target positions; this ensures the end effector is displayed if the motors stop moving despite not reaching the target angle positions.

## RESULTS

Five angle combinations are tested in both the MATLAB and Webots implementations. The results of which are shown below in Table 2:

| *Test* | $\theta_1$ | $\theta_2$ | $\theta_3$ | MATLAB Result 2.d.p (x,y,z) | Webots Result 2.d.p (x,y,z) | Difference |
|---|---|---|---|---|---|---|
| *1* | 0 | 0 | 0 | (0,0,1.2) | (0,0,1.2) | 0 |
| *2* | 0.5 | 0.3 | -0.2 | (0.11, 0.06, 1.18) | (0.11,0.06,1.18) | 0 |
| *3* | 1 | -0.4 | 0.6 | (-0.02, -0.03, 1.17) | (-0.02, -0.03, 1.17) | 0 |
| *4* | -0.7 | 0.8 | -0.5 | (0.26, -0.21, 1.09) | (0.26, -0.21, 1.09) | 0 |
| *5* | 1.2 | 0.2 | 0.4 | (0.10,0.27, 1.12) | (0.10,0.27, 1.12) | 0 |

**Table 2 - DH Forward Kinematics MATLAB vs Webots Results Table**

As indicated above in Table 2, there was no measured difference across all measured positions. This indicates a 100% success rate resulting in a perfect DH implementation in MATLAB with subsequent Webots measurements matching. One caveat to this is that the measured positions are rounded to two decimal places, which may hide inaccuracy. For the purposes of the project however, this is an acceptable level of accuracy.

# TASK 3 - INVERSE KINEMATICS IMPLEMENTATION

The inverse kinematic formula for the robot can be derived mathematically using the link lengths of each link as well as the pedestal height.

## MATLAB IMPLEMENTATION

The inverse kinematic function initially takes an inputted target position in a 3 dimensional space, in this case the x, y, z coordinates the robot needs to move to. As a result, the function outputs the target joint rotation angles for each link. This is achieved using a set of mathematical operations. This MATLAB function is shown below in Figure 4:

```
function [theta1, theta2, theta3] = inverse_k(x, y, z)

% Link Dimensions
ped_offset = 0.1;
a1 = 0.4;
a2 = 0.3;
a3 = 0.4;

% Base rotation (link 1)
theta1 = atan2(y, x);

% Reduce to 2D Plane
hor_distance = sqrt(x^2 + y^2);
vert_distance = z - ped_offset - a1;

% Distance
diagonal_distance = (hor_distance^2 + vert_distance^2 - a2^2 - a3^2) / (2 * a2 * a3);
diagonal_distance = max(-1, min(1, diagonal_distance));

% Elbow (link 3)
theta3 = acos(diagonal_distance);

% Shoulder (link 2)
theta2 = atan2(hor_distance, vert_distance) - atan2(a3 * sin(theta3), a2 + a3 * cos(theta3));

end
```

**Figure 4 - Inverse Kinematic MATLAB Function**

The function works according to the following mathematical methods, guided by the methodology discussed by Atique and Ahad (Atique & Ahad, 2014), referring to the link 1 joint, link 2 joint and link 3 joint as the base, shoulder and elbow respectively:

1. The robot dimensions are inputted into the function i.e. the height of the pedestal and the link lengths

2. "theta1 = atan2(y, x);" ascertains the rotation of the base (link 1) so that the robot is facing the target position
3. At this point, the mathematical problem has now changed from a 3 degree of freedom RRR to a two degree of freedom RR which can be solved using inverse kinematics:
    a. Solve horizontal distance from target: "hor_distance = sqrt(x^2 + y^2)"
    b. Solve vertical distance from target (relative to the shoulder joint): "vert_distance = z - ped_offset - a1;"
    c. Solve diagonal distance from target utilising law of cosines given a point and two sides (link 2 and 3 link lengths): "diagonal_distance = (hor_distance^2 + vert_distance^2 - a2^2 - a3^2) / (2 * a2 * a3), diagonal_distance = max(-1, min(1, diagonal_distance))"
    d. Using diagonal distance from target, ascertain elbow angle (link 3) assuming the elbow bends downwards: "theta3 = acos(diagonal_distance);"
    e. Find shoulder angle (link 2) adjusting for elbow angle (link 3): "theta2 = atan2(hor_distance, vert_distance) - atan2(a3 * sin(theta3), a2 + a3 * cos(theta3));"

## WEBOTS IMPLEMENTATION

The Webots controller ("move_by_arm_end_effector.m") implementation utilises the MATLAB implementation to calculate rotation angles for the robot given the end effector position, with the same angle limits as discussed in Task 2. The result is a 3 degree of freedom robotic arm that can position its end effector at a given target position. Aside from using calculated angles, the movement of the robot is identical to the implementation highlighted in Task 2.

## RESULTS

Five end effector positions are tested in both the MATLAB and Webots implementations. The results of which are shown below in Table 3:

| *Test* | **Target End Effector Position (x, y, z)** | **MATLAB Inverse Kinematic Result ($\theta_1$, $\theta_2$, $\theta_3$) 2.dp** | **Webots Recorded End Effector Position (x,y,z) 2.dp** | **Difference ($x_1$-$x_2$, $y_1$-$y_2$, $z_1$-$z_2$)** |
|---|---|---|---|---|
| *1* | (0.0, 0.0, 1.2) | (0, 0, 0) | (0.0, 0.0, 1.2) | (0,0,0) |
| *2* | (0.2, 0.2, 0.8) | (0.79, -0.40, 1.91) | (0.2, 0.2, 0.8) | (0,0,0) |
| *3* | (-0.3, 0.1, 0.7) | (2.82, -0.25, 2.05) | (-0.3, 0.1, 0.7) | (0,0,0) |
| *4* | (0.1, -0.4, 0.6) | (-1.33, 0.21, 1.87) | (0.1, -0.4, 0.6) | (0,0,0) |
| *5* | (-0.2, -0.2, 0.9) | (-2.36, -0.34, 1.61) | (-0.2, -0.2, 0.9) | (0,0,0) |

**Table 3 - Inverse Kinematics MATLAB vs Webots Results Table**

As indicated above in Table 3, there was no measured difference across all measured positions. This indicates a 100% success rate resulting in a perfect Inverse Kinematic implementation in MATLAB with subsequent Webots confirmation. One caveat to this is that the measured positions are rounded to two decimal places, which may hide inaccuracy. For the purposes of the project however, this is an acceptable level of accuracy.

# TASK 4 - AI-BASED INVERSE KINEMATICS USING

Using artificial intelligence based models can allow for the prediction of values for both inverse and forward kinematics of the robotic arm. The calculation of joint variables for the purposes of inverse kinematics is "complex, difficult and computationally expensive" (Sharkawy & Khairullah, 2023), and as such, utilising a neural network may prove more efficient despite potential inaccuracy in real time solutions. As such, a fully connected dense neural network is implemented using Python in Google Collab to provide comparison with the previously described MATLAB implementation

## NEURAL NETWORK IMPLEMENTATION

The implementation shall utilise the same method of computing the inverse kinematic given a target position as Task 3's MATLAB implementation. This has been contained within the function "inverse_k(x,y,z)" and utilised in the creation of a dataset as shown below in Figure 5:

```python
rows = []
size = 10000

#Create Raw Data
output_file_raw = "inverse_kinematics_data.csv"
if not os.path.exists(output_file_raw):
    while len(rows) < size:
        x = np.random.uniform(-1, 1)
        y = np.random.uniform(-1, 1)
        z = np.random.uniform(0, 1)

        try:
            theta1, theta2, theta3 = inverse_k(x, y, z)
            rows.append({
                "x": x,
                "y": y,
                "z": z,
                "theta1": theta1,
                "theta2": theta2,
                "theta3": theta3
            })

        except ValueError as e:
            print(f"{e}")


    df = pd.DataFrame(rows)
    df.to_csv(output_file_raw, index=False)
```

**Figure 5 - Dataset Creation**

```python
#Create Normalised Data
output_file_normalised = "inverse_kinematics_data_normalised.csv"
rows = []
if not os.path.exists(output_file_normalised):
    for index, row in df.iterrows():
        x = (row["x"] + 1) / 2
        y = (row["y"] + 1) / 2
        z = row["z"]

        theta1 = (row["theta1"] + np.pi) / (2 * np.pi)
        theta2 = (row["theta2"] + np.pi/2) / np.pi
        theta3 = row["theta3"] / np.pi

        rows.append({
            "x": x,
            "y": y,
            "z": z,
            "theta1": theta1,
            "theta2": theta2,
            "theta3": theta3
        })

    df = pd.DataFrame(rows)
    df.to_csv(output_file_normalised, index=False)

else:
    df = pd.read_csv(output_file_normalised)
```

**Figure 6 - Dataset Normalisation**

As shown above in Figure 5, a dataset of 10000 random samples is generated utilising the inverse kinematic function. Utilising random values of x (between -1 and 1), random values of y (between -1 and 1) and random values of z (between 0 and 1), accurate theta values are calculated and contained within a csv file. Subsequently, as shown in Figure 6, the dataset is normalised to values between 1 and 0 and produced as a separate csv file. The dataset is then loaded as a pandas data frame.

```python
#Create or Load Neural Network
model_file = "ik_model.keras"
if not os.path.exists(model_file):

    #Create Training and Testing Data
    x = df[["x", "y", "z"]].values
    y = df[["theta1", "theta2", "theta3"]].values
    x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.2, random_state=42)

    #Build Model
    model = keras.Sequential([
        keras.layers.Input(shape=(3,)),
        keras.layers.Dense(64, activation='relu'),
        keras.layers.Dense(64, activation='relu'),
        keras.layers.Dense(64, activation='relu'),
        keras.layers.Dense(3, activation='tanh')
    ])

    #Compile Model
    model.compile(
        optimizer='adam',
        loss='mse',
        metrics=['mae']
    )

    #Train Model
    history = model.fit(
        x_train, y_train,
        validation_data=(x_test, y_test),
        epochs=100,
        batch_size=32,
        verbose=1
    )

    #Evaluate Model
    loss, mae = model.evaluate(x_test, y_test)
    print("Test Loss:", loss)
    print("Test MAE:", mae)

    #Save Model
    model.save("ik_model.keras")

else:
    model = keras.models.load_model(model_file)
```

**Figure 7 - Inverse Kinematic Neural Network Model**

Figure 7 showcases the creation of the neural network model. The model utilises the normalised data, splitting 80% for training and 20% for testing. The model intakes the x, y and z values of each sample as inputs in the keras sequential input layer, with three dense output layers. The architectures three hidden dense layers each contain 64 neurons with ReLu

activation functions with a final output layer using tanh activation. Training is subsequently conducted using 100 epochs and values for Mean Squared Error and Mean Absolute Error are obtained as 0.002 and 0.02 respectively, indicating high model accuracy. Finally, the model is stored in the Keras file format.

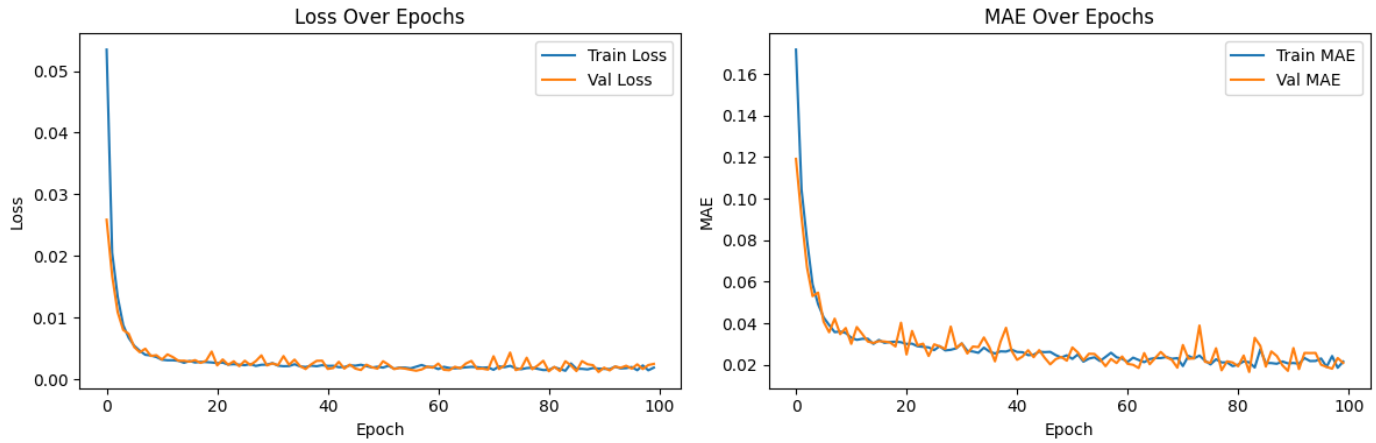


**Figure 8 - Neural Network Training History**

Figure 8 above shows the training history of the neural network. No significant overfitting or underfitting is observed by the training and validation curves in the loss plot. The MAE plot also shows stable training curves, indicating predictions are consistently accurate and they converge to around 0.02 matching the reported Mean Absolute Error value. A plateau observed in both graphs indicates that training could have ceased earlier.

## COMPARISON WITH TASK 3 MATLAB IMPLEMENTATION

| *Test* | **Target End Effector Position (x, y, z)** | **MATLAB Inverse Kinematic Result ($\theta_1$, $\theta_2$, $\theta_3$) 2.dp** | **Neural Network Inverse Kinematic Result ($\theta_1$, $\theta_2$, $\theta_3$) 2.dp** | **Difference ($x_1$-$x_2$, $y_1$-$y_2$, $z_1$-$z_2$)** |
|---|---|---|---|---|
| *1* | (0.0, 0.0, 1.2) | (0, 0, 0) | (0.20,-0.27,0.89) | (-0.20,0.27,-0.89) |
| *2* | (0.2, 0.2, 0.8) | (0.79, -0.40, 1.91) | (0.74,-0.41,1.92) | (0.05,0.01,-0.01) |
| *3* | (-0.3, 0.1, 0.7) | (2.82, -0.25, 2.05) | (2.75,-0.26,2.05) | (0.07,0.01,0.00) |
| *4* | (0.1, -0.4, 0.6) | (-1.33, 0.21, 1.87) | (-1.38,0.17,1.86) | (0.05,0.04,0.01) |
| *5* | (-0.2, -0.2, 0.9) | (-2.36, -0.34, 1.61) | (-2.33,-0.32,1.59) | (-0.03,-0.02,0.02) |

**Table 4 - Comparison Table between Task 3 Inverse Kinematic Implementation Results and Neural Network Implementation**

As shown in Table 4 above, results are within +-0.1 radians of the target theta values aside from Test 1. This indicates that the model performance is acceptable and accurate except for the anomalous test 1 result. Furthermore, as test 1 is indicative of the robotic manipulator at rest, this deviation may be due to the model not accounting for the zero position of the robot. A correction could be made to include the zero position in the dataset as a known position whereby theta values are definitively (0,0,0) at this end effector position.

# TASK 5 PICK-AND-PLACE TASK USING INVERSE KINEMATICS

The core feature of a 3 degrees of freedom robotic manipulator is to perform functions. The inverse kinematic implementation can be utilised to achieve this aim. As such, implementing a pick and place methodology for the robotic system can showcase the ability of the inverse kinematic method designed.

## GRABBER IMPLEMENTATION

### OVERVIEW

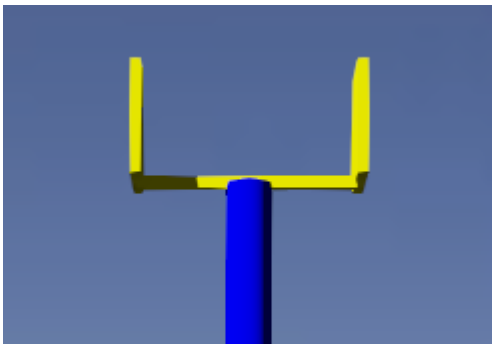

**Figure 9 - Grabber Mechanism of Robotic Manipulator**

A claw-like structure is placed at the end effector position of the robotic manipulator as shown in Figure 9. This consists of a base and two fingers which have the aim of closing to grasp objects.

The implementation is constructed in Webots by defining two slider joints, each controlling a finger of the grabber. The fingers are modelled as thin symmetrical boxes with a large surface area to allow for easier grabbing of objects. The joints are driven by linear motors which allow for the sliding mechanism of the grabber allowing the fingers to slide shut and close, fulfilling the grabbing procedure. Figure 10 below illustrates different stages of the grabbing process:


**Figure 10 - Different Stages of Pick and Place Implementation**

### MATLAB CONTROLLER IMPLEMENTATION

The controller implementation of the pick-and-place mechanism follows similar procedures to the aforementioned implementations. Utilising identical movement procedures, the robotic arm finds the target box' position and utilises the inverse kinematic implementation to orient itself to slightly above the target position. This places the target within the fingers of the gripper. Figure 11 below illustrates the core functionality of the closing and opening of the fingers to grip the target object:

```
%Gripper Functions
function open_gripper()
  timestep = wb_robot_get_basic_time_step();

  %Get Gripper Motors
  right_gripper_motor = wb_robot_get_device('gripper_right_motor');
  left_gripper_motor  = wb_robot_get_device('gripper_left_motor');

  %Set Gripper Force
  grip_force = 10
  wb_motor_set_available_force(right_gripper_motor, grip_force)
  wb_motor_set_available_force(left_gripper_motor, grip_force)

  % Get Gripper sensors
  right_gripper_sensor = wb_robot_get_device('gripper_right_sensor');
  left_gripper_sensor  = wb_robot_get_device('gripper_left_sensor');

  % Enable sensors
  wb_position_sensor_enable(right_gripper_sensor, timestep);
  wb_position_sensor_enable(left_gripper_sensor, timestep);

  wb_robot_step(timestep);

  opened_position = 0
  gripper_motor_speed = 0.1
  wb_motor_set_velocity(right_gripper_motor, gripper_motor_speed);
  wb_motor_set_velocity(left_gripper_motor, gripper_motor_speed);
  wb_motor_set_position(right_gripper_motor, opened_position);
  wb_motor_set_position(left_gripper_motor, opened_position);

  time = 0;
  while wb_robot_step(timestep) ~= -1
    time = time + timestep/1000;
    if time>=2
      break
    end
  end
```

```
function close_gripper()
  timestep = wb_robot_get_basic_time_step();

  %Get Gripper Motors
  right_gripper_motor = wb_robot_get_device('gripper_right_motor');
  left_gripper_motor  = wb_robot_get_device('gripper_left_motor');

  %Set Gripper Force
  grip_force = 10
  wb_motor_set_available_force(right_gripper_motor, grip_force)
  wb_motor_set_available_force(left_gripper_motor, grip_force)

  % Get Gripper sensors
  right_gripper_sensor = wb_robot_get_device('gripper_right_sensor');
  left_gripper_sensor  = wb_robot_get_device('gripper_left_sensor');

  % Enable sensors
  wb_position_sensor_enable(right_gripper_sensor, timestep);
  wb_position_sensor_enable(left_gripper_sensor, timestep);

  wb_robot_step(timestep);

  closed_position = 0.1
  gripper_motor_speed = 0.1
  wb_motor_set_velocity(right_gripper_motor, gripper_motor_speed);
  wb_motor_set_velocity(left_gripper_motor, gripper_motor_speed);
  wb_motor_set_position(right_gripper_motor, closed_position);
  wb_motor_set_position(left_gripper_motor, closed_position);

  time = 0;
  while wb_robot_step(timestep) ~= -1
    time = time + timestep/1000;
    if time>=2
      break
    end
  end
end
```

**Figure 11 - Opening and Closing Gripper Functions**

The gripper opens or closes its fingers with a motor strength of 10 as well as a speed of 0.1 using predefined positions for the fingers to reach (fingers touching or reaching the end of the base). With a time, limit of 2 seconds, the gripper will close or open its fingers allowing it to grasp the target object. Subsequently, it will move to the zero position of the robot, to mitigate collision with the ground and demonstrate efficacy of the implementation before moving to the target position and placing the object.

## REFERENCES

Atique, M. M. U. & Ahad, M. A. R., 2014. *Inverse Kinematics Solution for a 3DOF Robotic Structure using Denavit-Hartenberg Convention*. Dhaka, 3rd International Conference on Informatics, Electronics & Vision.

Cyberbotics, 2025. *Webots Open Source Robot Simulator*. [Online]
Available at: https://cyberbotics.com/
[Accessed 05 04 2026].

Prada, E., Murali, S., Miková, Ľ. & Ligušová, J., 2020. Application of Denavit Hartenberg Method in Service Robotics. *Acta Mechatronica,* 5(4), p. 47–52.

Sharkawy, A.-N. & Khairullah, S. S., 2023. Forward and inverse kinematics solution of a 3-DOF articulated robotic manipulator using artificial neural network. *International Journal of Robotics and Control Systems,* 3(2), p. 330–353.